

# RT-DETR

# Paper Review

DETRs Beat YOLOs on Real-time Object Detection - Yian Zhao et al.

연구원 김재현  
E. [jh.kim@zipida.com](mailto:jh.kim@zipida.com)

# CONTENTS

01

## YOLO

- Anchor
- NMS

02

## DETR

- Transformer

03

## RT-DETR

- Efficient Hybrid Encoder
- Uncertainty-minimize  
Query Selection

04

## Code implement

Paper  
Review

# PART 1. YOLO

Anchor Box

NMS

# Anchor Box

개요: 잠재적으로 객체가 있을만한 형태의 박스의 대략적 형태를 사전에 생성하여 예측 속도와 정확도를 늘리는 전략

장점: 각 epoch마다 Anchor Box의 형태를 Loss와 학습을 통해 조금씩 변경해나가서 BBox로 사용함

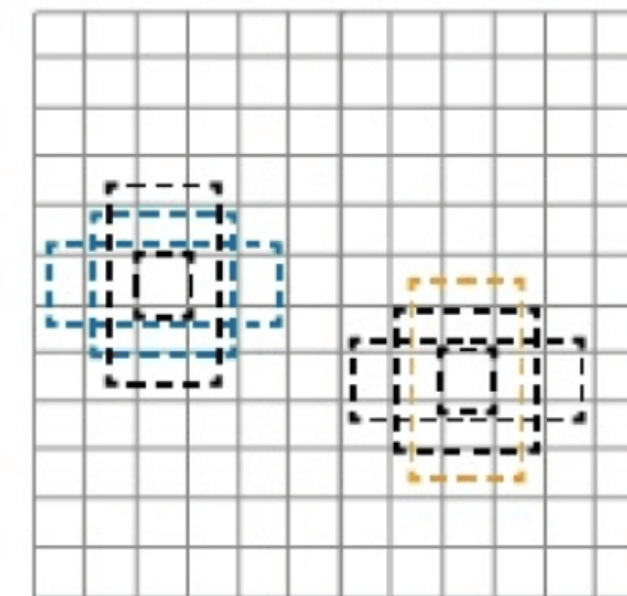
→ 일반적인 형태의 객체에 대해 Detecting이 용이하고 수렴이 빠름

단점: BBox가 사전생성된 Anchor Box의 형태에 종속적임,  
각 도메인마다 최적 Anchor 값을 직접 파라미터로 넣어줘야 하기 때문에  
도메인 간 이식이 어려움

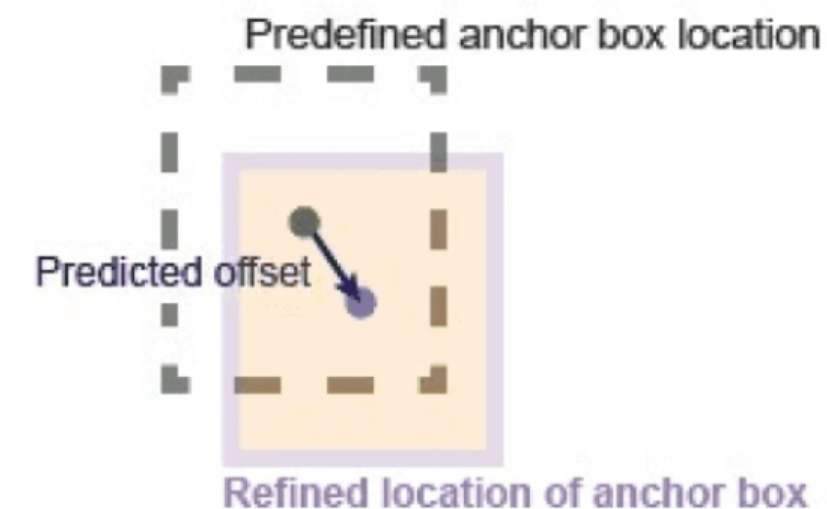
→ 사전 생성된 Anchor Box에 없는 형태의 객체에 대해 예측력 저하



Ground truth image and bounding boxes



Anchor boxes at each predefined location in each feature map



참고문헌)

Redmon, Joseph, et al. 2016. "You Only Look Once: Unified, Real-Time Object Detection."

Redmon, Joseph, et al. 2018. "YOLOv3: An Incremental Improvement."

출처)

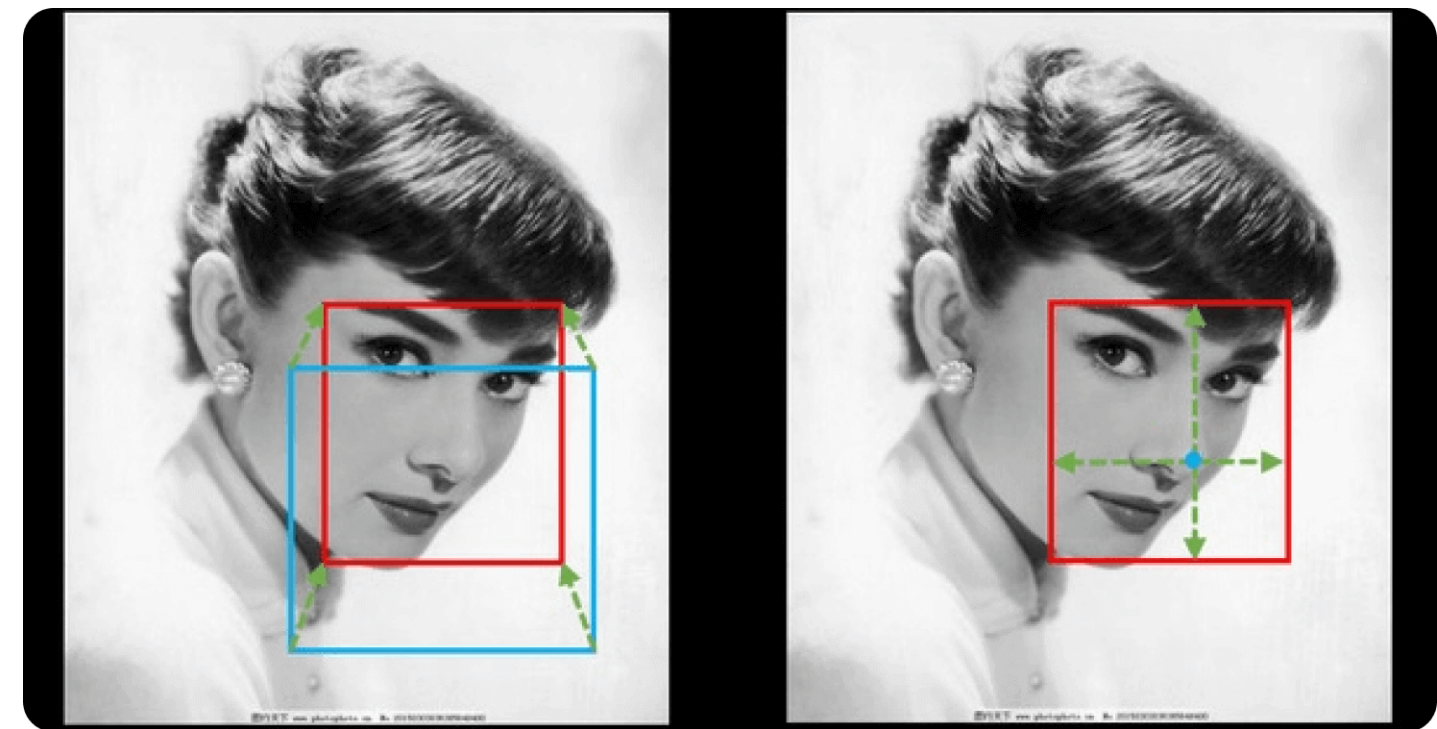
<https://www.ultralytics.com/ko/blog/benefits-ultralytics-yolo11-being-anchor-free-detector>

# Anchor-free

개요: Anchor Box를 사용하지 않고 BBox의 크기를 직접 회귀해서 예측하는 전략 (YOLOv8 부터 사용)

장점: 사전 생성된 Anchor의 형태에 종속적이지 않으므로 독특한 형태의 객체도 준수하게 Detecting 가능함, 작거나 불규칙적이거나 가려진 객체도 탐지 가능성이 증가

단점: 하나의 Cell에 여러 객체가 겹쳐있거나 밀집되어 있으면 중심점 (center point)이 겹쳐져서 객체의 분류가 약해짐, 거리를 회귀로 측정하기 때문에 특이 형태에 취약



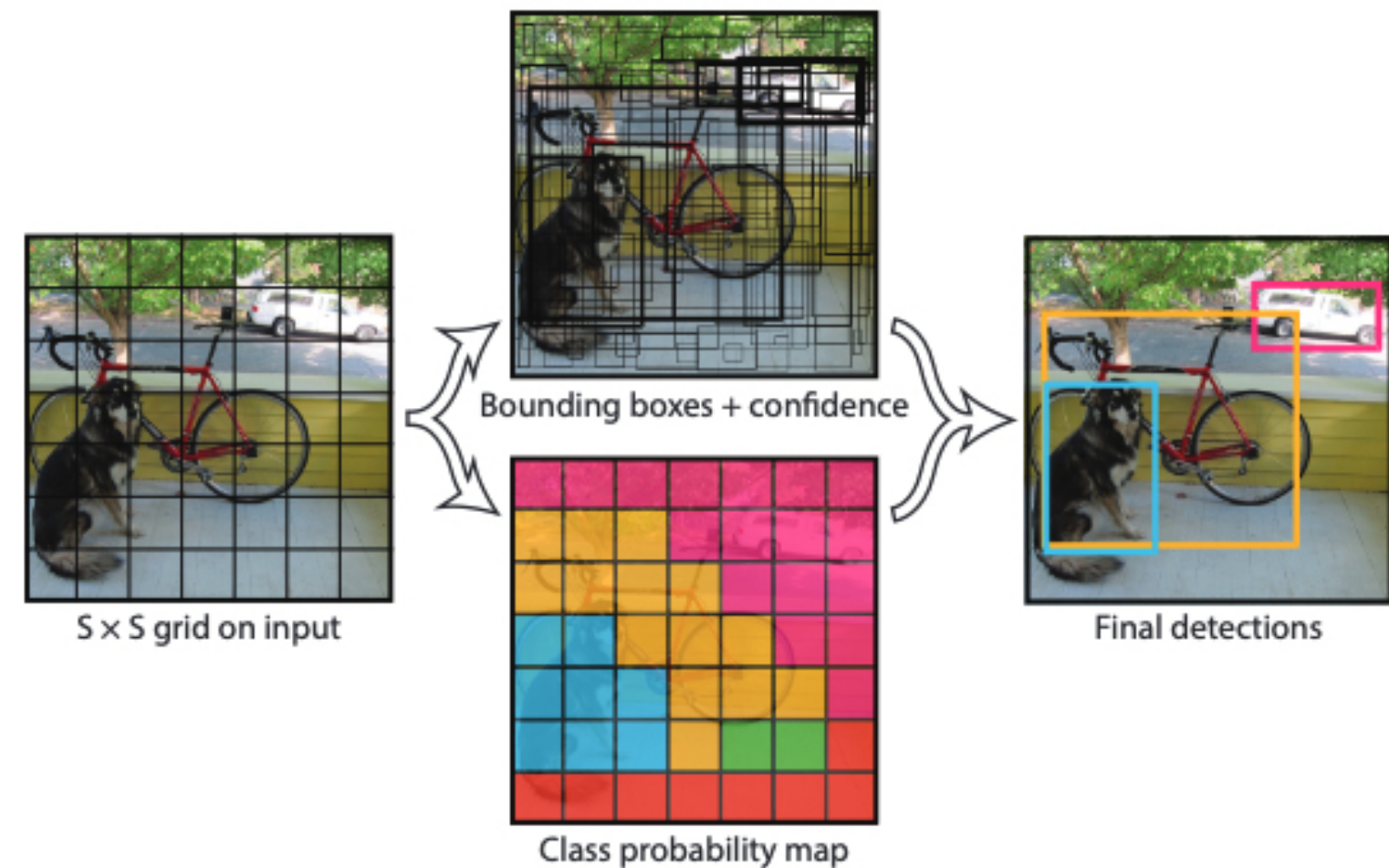


# NMS

개요: 예측된 바운딩 박스를 두 가지 Confidence Score와 IoU (Intersection over Union) 임계값을 기반으로 의사 결정을 내리는 전략

장점: 간단하고 직관적인 구현, 설정된 임계값에 따라 정확하게 BBox가 소거 및 정리됨, 모델 구조와 독립적으로 후처리 할 수 있어서 쉽게 사용가능함

단점: 임계값 이상 겹쳐있는 같은 라벨의 객체가 있으면 서로 다른 객체여도 제거되기 때문에 recall이 하락함, 디텍터 모델 이후에 후처리를 하나의 과정으로 추가하는 방식이기 때문에 복잡도 증가, End-to-End 불가



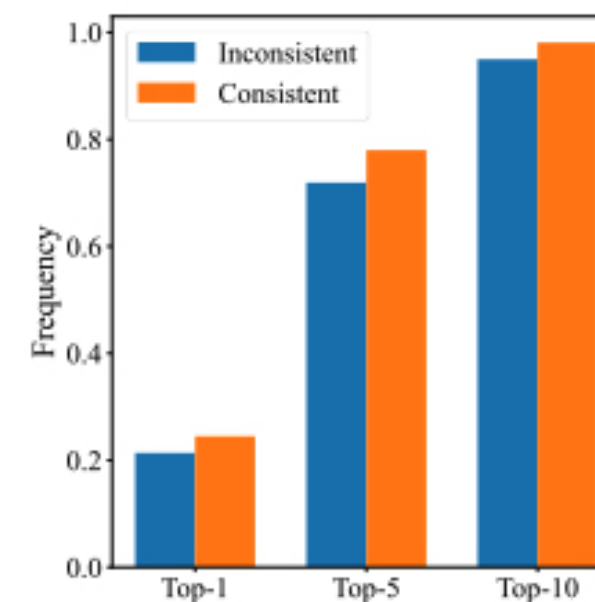
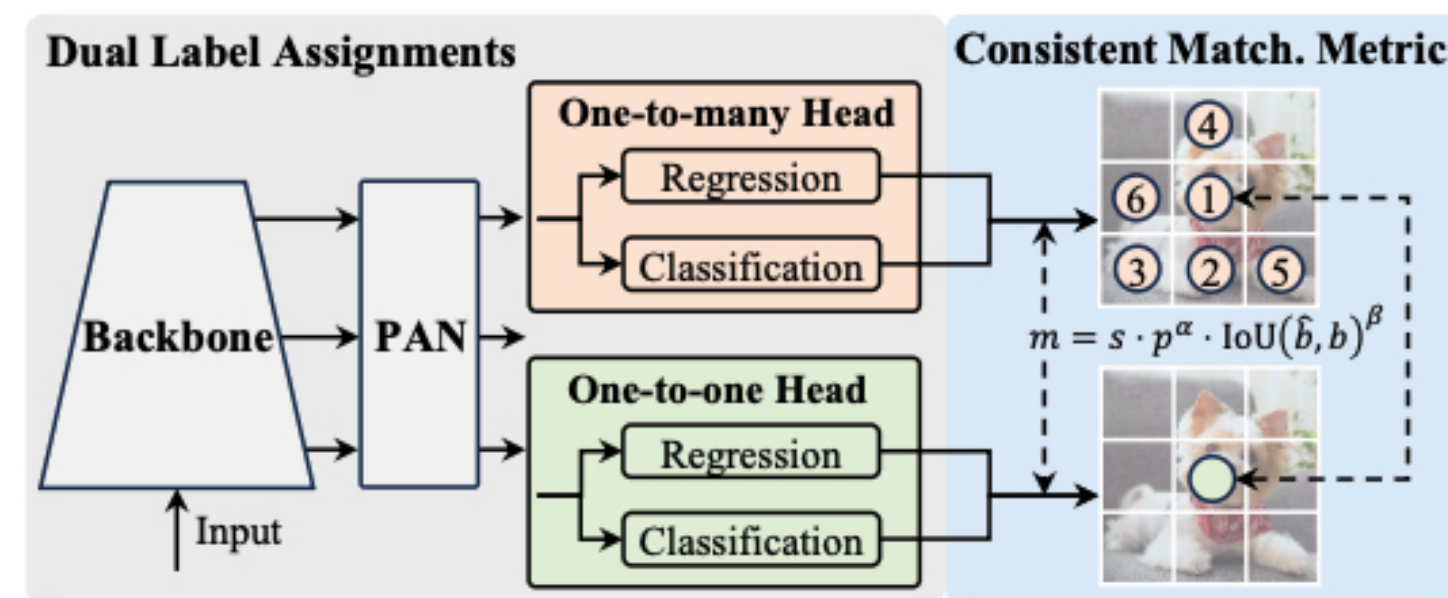


# NMS-free

개요: 추론 단계에서 NMS를 제거하고 훈련에서부터 하나의 GT ↔ 하나의 예측(one-to-one) 매칭을 강하게 걸어 중복 없는 출력을 직접 학습하는 전략(YOLOv10 부터 사용)

장점: 추론 이후 NMS 후처리가 제거되었기 때문에 End-to-End 학습 가능, 겹쳐있거나 가려진 물체에 대해서도 one-to-one 매칭으로 인해 디텍팅 성능이 개선됨

단점: 후처리 파이프라인을 제거하였기 때문에 모델 자체에서 객체당 BBox 예측 매커니즘이 필요하게 되어 모델 구성 난이도가 증가함, 학습 초기 쿼리-객체 매칭 난이도가 증가하고 객체 매칭이 잘 이루어지지 않으면 Gradient 불안정



참고문헌)

Hussain, Muhammad. 2024. "YOLOv5, YOLOv8 and YOLOv10: The Go-To Detectors for Real-time Vision."

Paper  
Review

# PART 2. DETR

DEtecting TRansformer

Transformer

Detector



# DETR

요약)

CNN, Transformer Architecture를 활용한 완전 End-to-End 디텍터

특징)

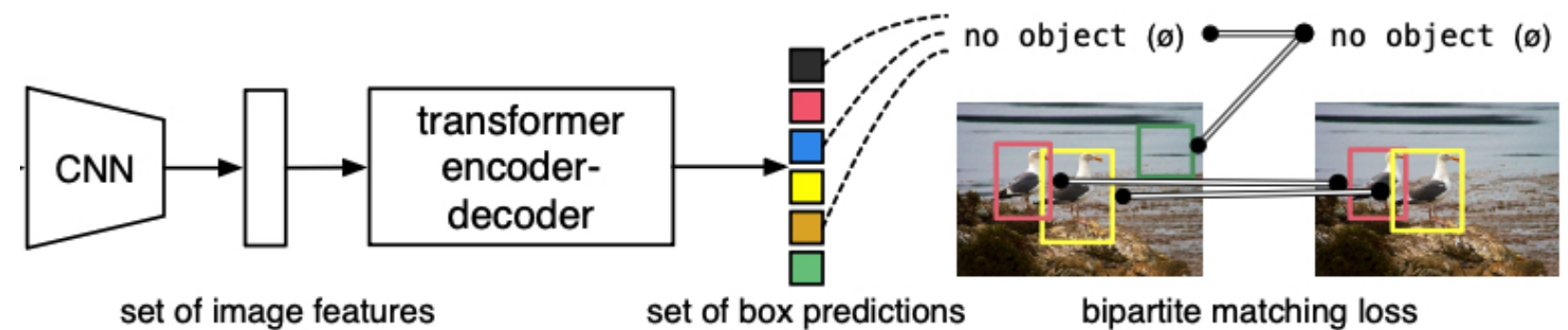
Encoder input : CNN 백본을 거쳐 나온 Feature Map을  $(HW) * C$ 로 reshape해서 Pos-Embedding 한 값

Encoder output : Decoder의 두번째 Cross-Attention 의 K, V

Decoder input : 하이퍼 파라미터로 정한 개수의 슬롯 벡터

Decoder output : 고정 차원의 벡터값

→ 이후 MLP(Cls) head와 MLP(Box) head를 거쳐 나온 값이 클래스와 BBox 좌표



참고문헌)

Carion, Nicolas, et al. 2020. "End-to-End Object Detection with Transformers."

# DETR Architecture

백본)

input  $[B, C, H, W] \rightarrow \text{CNN(ResNet50)} \rightarrow [B, 2048, H/32, W/32]$   
 $\rightarrow \text{Conv}(1*1) \rightarrow \text{output } [B, 256, H/32, W/32]$  (최초 논문 기준)

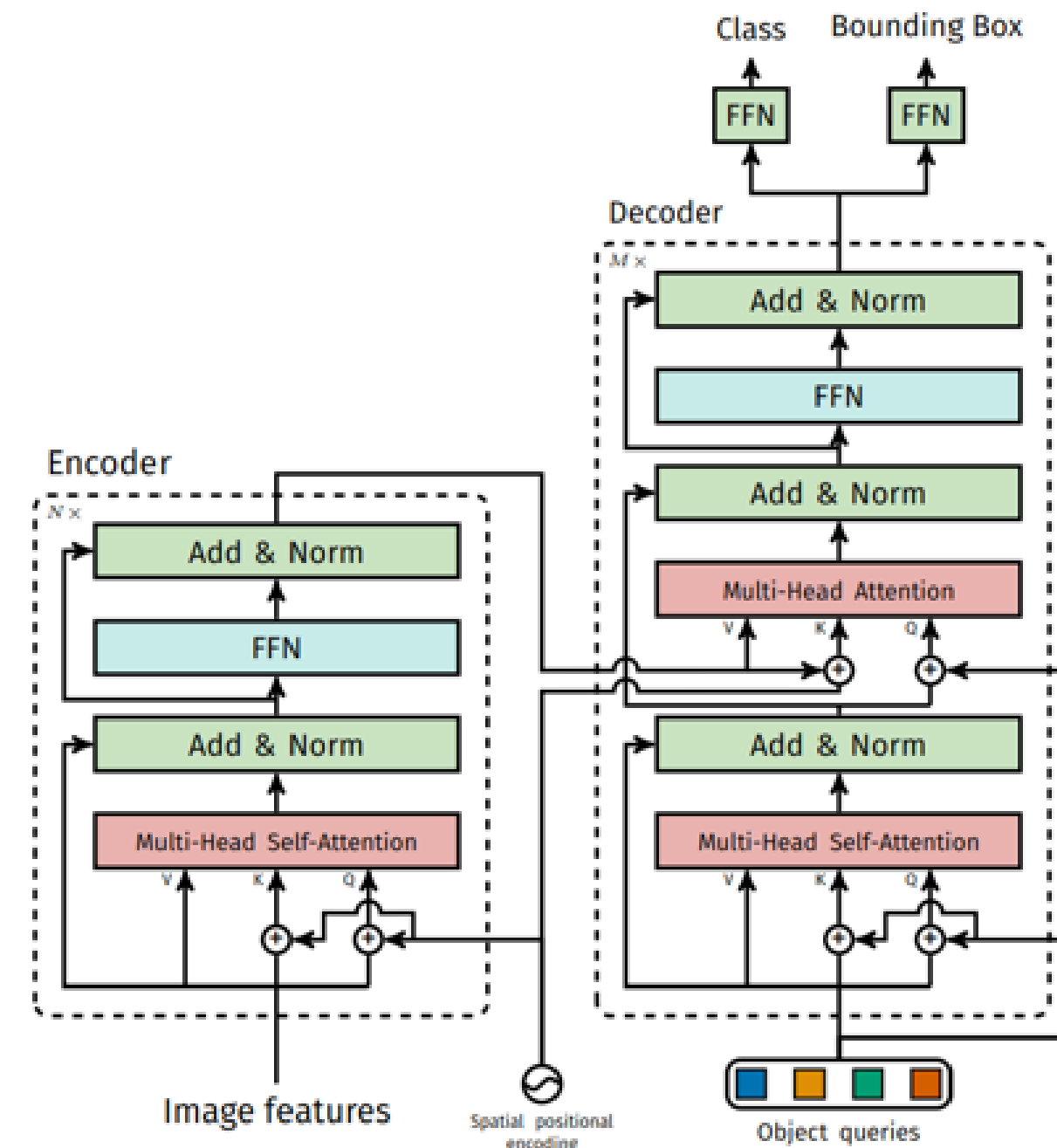
Transformer)

Encoder: reshape  $[B, (H/32*W/32), 256] \rightarrow \text{pos\_embedding}$   
 (add)  $\rightarrow \text{Transformer Encoder} \rightarrow \text{output (KV cache)}$

Decoder: input  $[B, N(100), 256] \rightarrow \text{Multi-Head Self-Attention} \rightarrow$   
 $\text{Multi-Head Cross-Attention}(Q=\text{decoder input, K,V=encoder output}) \rightarrow \text{FFN} \rightarrow \text{output } [B, N(100), 256]$

Head)

output: MLP\_cls와 MLP\_box를 거쳐  $[B, N, C] / [B, N, 4]$



# Hungarian Loss

Ground Truth Box와 예측된 Bounding Box를 1:1 매칭하여 Train Loss 계산

$$\mathcal{L}_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=1}^N \left[ -\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \right]$$

클래스 예측 손실

$(c_i)$  = i번째 실제 객체의 클래스 라벨

만약 실제 객체가 없는 경우(배경)에는  $\emptyset$ (no object)으로 표시

$\hat{p}_{\hat{\sigma}(i)}(c_i)$  =  $\hat{\sigma}(i)$ 번째 예측이 실제  $(c_i)$  클래스일 확률

음의 로그 우도 손실(negative log-likelihood loss)로, 모델이 실제 클래스를 정확하게 예측할수록 손실이 작아지는 방식

실제 객체가 존재하는 경우 1,  
실제 객체가 존재하지 않을 경우 0

BBox 예측 손실

GT Box와 BBox의 거리를 L1으로 측정

L1 loss와 **Generalized IoU loss**를 선형 결합

$$\bullet \text{ } GIoU = IoU - \frac{C \setminus (A \cup B)}{|C|}$$



$$\bullet \text{ } -1 < GIoU < 1 \text{ } (0 < IoU < 1)$$

$$\bullet \text{ } Loss_{GIoU} = 1 - GIoU, 0 < Loss_{GIoU} < 2$$

GIoU Loss

GT, BBox를 모두 포함하는 가장 작은 박스(convex hull) C를 만들어 두 박스가 겹쳐있지 않더라도 거리와 이동해야 하는 방향에 대한 정보를 Loss로 제공할 수 있음

$$\text{Box Loss} = \lambda_{\text{iou}} \mathcal{L}_{\text{iou}}(b_i, \hat{b}_{\sigma(i)}) + \lambda_{\text{L1}} \|b_i - \hat{b}_{\sigma(i)}\|_1$$

GIoU Loss와 L1 Loss를 선형으로 결합한 형태

# DETR 한계

## 1) 느린 학습 / 비효율적인 인코더

백본을 통과해 나온 feature map 전체를 트랜스포머에 넣어서 self-attention을 진행하기 때문에 학습이 오래걸림

작은 객체의 정보가 소실되었기 때문에 BBox 예측이 어려움

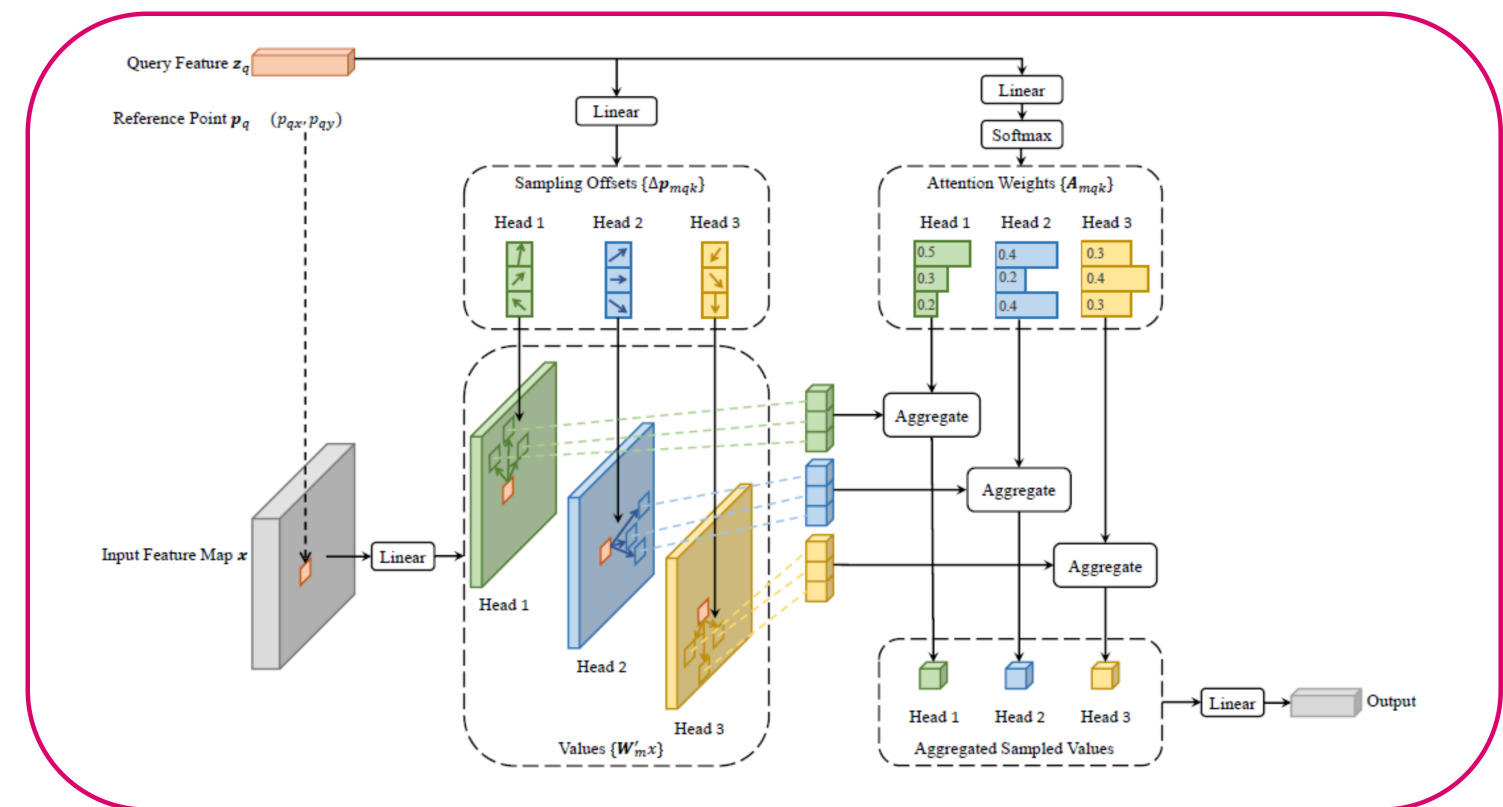
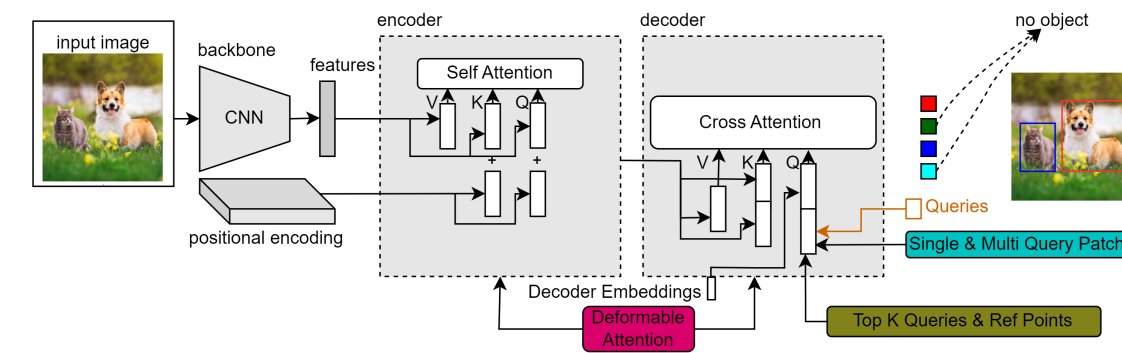
self-attention의 복잡도 =  $O((H'W')^2)$  (연산량 폭증)

→ 실시간 불가

## 2) 쿼리 초기화의 Uncertainty

각 슬롯 벡터가 정해진 GT Box없이 랜덤하게 객체를 담기 때문에 최초에 여러 슬롯이 하나의 객체를 담는다거나, 어떤 실제 객체는 담지 않는다거나 하는 문제가 있음

→ 훈련 초기 Cls와 IoU가 일치하지 않는 경우에 학습 속도 저하



Paper  
Review

# PART 3. RT-DETR

Real-Time DEtecting TRansformer

Real-Time

Uncertainty minimize  
query selection



# RT-DETR

요약)

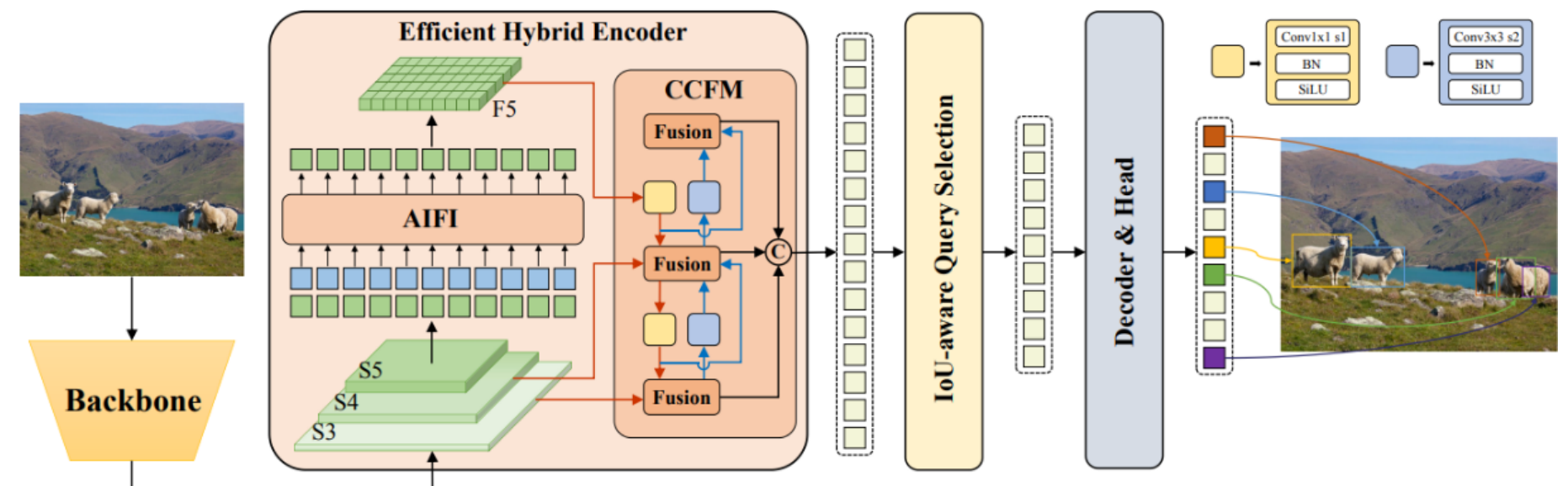
RT-DETR는 DETR의 느리고 불분명한 부분을 고쳐서, YOLO급 실시간 속도 + SOTA 성능을 뽑는 DETR

Efficient Hybrid Encoder)

백본의 output을 Residual Connection으로 고해상도 저의미도 학습  
백본의 output을 두 번 MaxPool 해서 더 작은 이미지를 self-attention  
(AIFI - Attention-based Intra-scale Feature Interaction)

IoU-aware Query Selection)

인코더 뒤에 분류 분포  $C(X)$ , 로컬라이제이션 분포  $P(X)$ 를 출력하는 작은  
헤드를 엮어 벡터 슬롯을 최적화



참고문헌)

Zhao, Yian, et al. 2024. "DETRs Beat YOLOs on Real-time Object Detection."

# Hybrid Encoder

## 1) AIFI(Attention-based Intra-scale Feature Interaction)

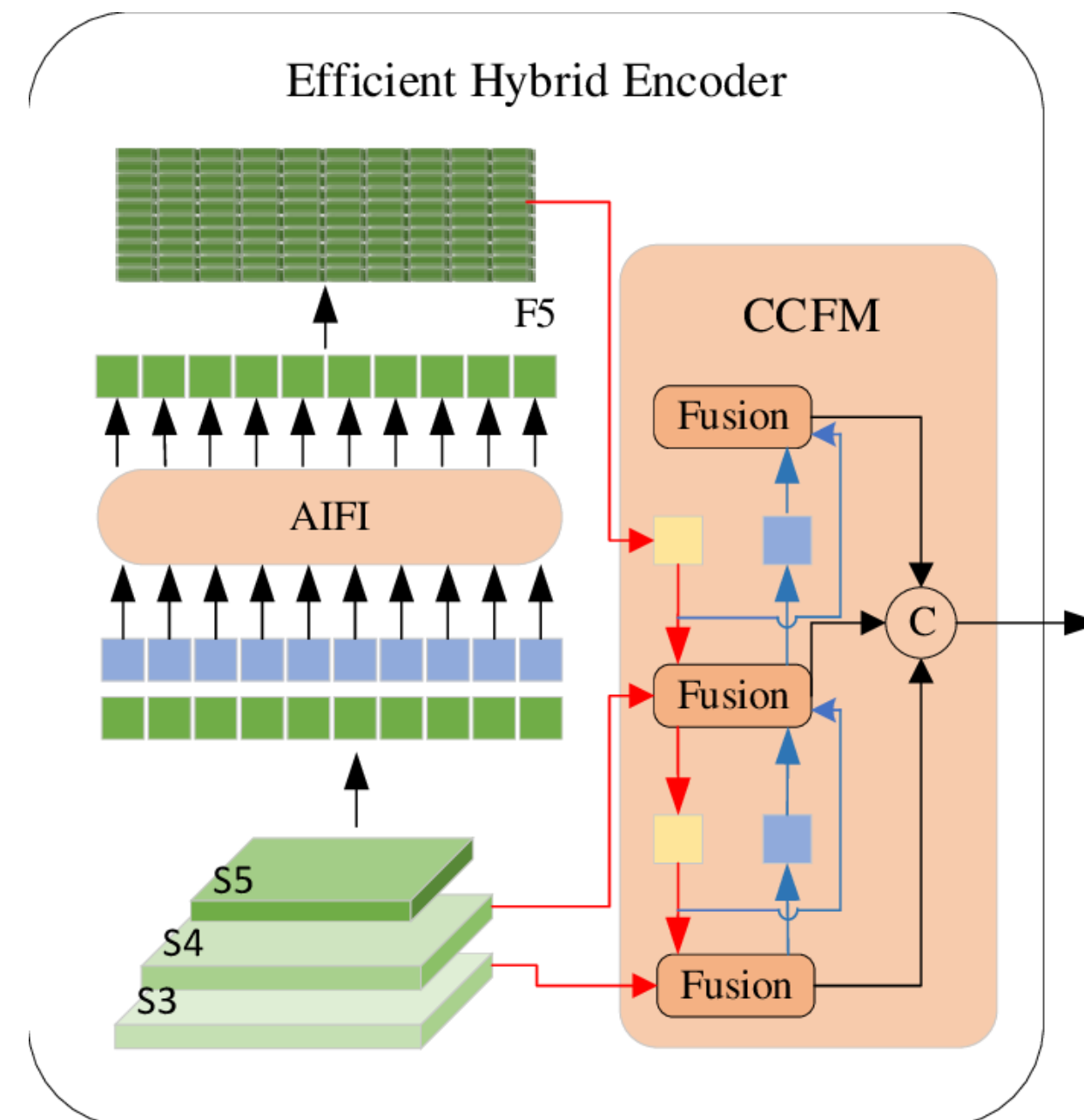
S3[B, 256, H/8, W/8], S4[B, 256, H/16, W/16], S5[B, 256, H/32, W/32] 에서 S5만 Self-Attention에 넣어서 전역 이미지에 대한 정보를 획득하고 CCFM으로 투입

효과: 고수준 특징(S5)에서만 intra-scale 상호작용을 수행하는 것이 불필요한 저수준 특징의 상호작용을 제거하고, 성능을 더 좋게 만듦

## 2) CCFM(CNN-based Cross-scale Feature-fusion Module)

AIFI를 통과한 F5를 CCFM에서 upsample + Residual Connection + downsampling 하여 고해상도 feature map도 이해할 수 있게 함  
세 개의 fusion된 feature map을 하나로 concat하여 IoU-aware Query Selection에 입력

효과: PANet-style의 CSF를 개선하여, 융합 경로에 여러 fusion block 을 삽입하여 Convolutional layer를 통해 인접한 스케일의 특징들을 효율적으로 융합하도록 설계





# RT-DETR 특징

## 1) 학습속도 개선

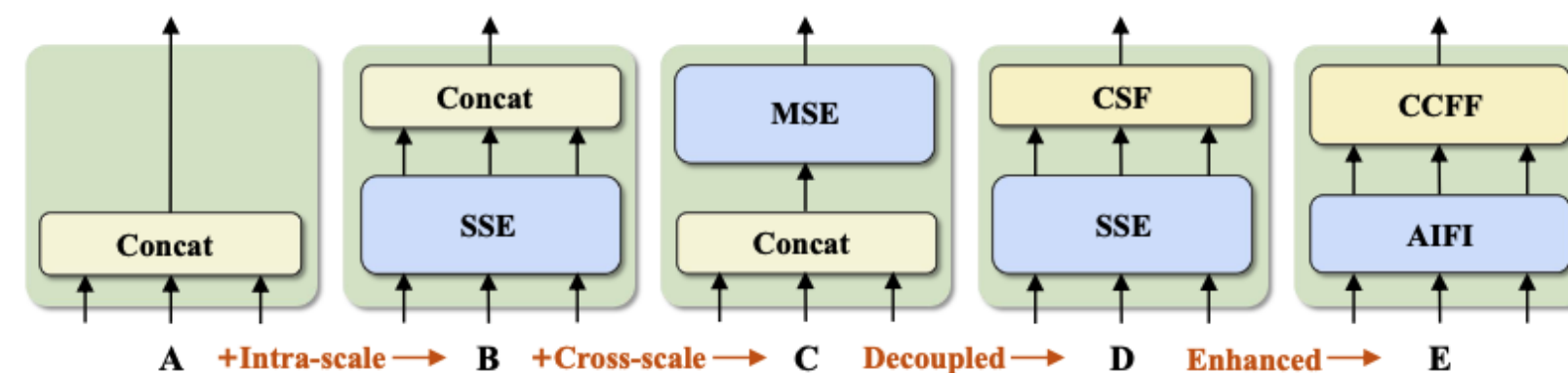
Uncertainty-Minimal Query Selection 으로 학습 초기 속도가 저하되던 문제점을 해소하였음

## 2) NMS 후처리 제거

후처리 과정을 제거하여 End-to-End로 동작하는 모델을 만들었고 이로 인해 실시간성을 높일 수 있었음

## 3) Transformer 의 높은 계산 비용 문제 해결

기존 DETR은 NMS가 없음에도 트랜스포머의 높은 연산량으로 인해 실시간 성능을 달성할 수 없었으나 Efficient Hybrid Encoder로 해결



B) SSE(Single-Scale Transformer Encoder)

하나의 feature map이 트랜스포머 인코더를 통과하는 구조(기본 DETR)

C) MSE(Multi-Scale Transformer Encoder)

여러 스케일의 feature map이 트랜스포머 인코더를 통과하는 구조

D) CSF(Cross-Scale Fusion)

여러 스케일의 intra-scale intersection은 SSE에서 이루어지고 여러 스케일의 특징들을 PANet 형태로 융합하는 구조

E) RT-DETR 구조

intra-scale 상호작용: AIFI(Attention-based Intra-scale Feature Interaction)

cross-scale 융합: CCFF (CNN-based Cross-scale Feature Fusion)

# PART 4. Code Implement

## 1) Backbone

```
def build_backbone(train_backbone: bool = False,
                  dilation: bool = False) -> nn.Module:
    """
    ResNet-50 백본을 생성하여 입력 피쳐와 마지막 피쳐를 반환하도록 구성한다.
    """
    kwargs = {"replace_stride_with_dilation": [False, False, dilation]}
    if _RESNET50_DEFAULT_WEIGHTS is not None:
        kwargs["weights"] = _RESNET50_DEFAULT_WEIGHTS
    else:
        kwargs["pretrained"] = True

    backbone = resnet50(**kwargs)
    _replace_bn_with_frozen(backbone)

    return_layers = {"layer4": "0"}
    if dilation:
        backbone.layer4[0].conv2.stride = (1, 1)
        backbone.layer4[0].downsample[0].stride = (1, 1)

    backbone = IntermediateLayerGetter(backbone, return_layers=return_layers)
    if not train_backbone:
        for param in backbone.parameters():
            param.requires_grad_(False)
    return backbone
```

## 2) Transformer

```
class SimpleTransformer(nn.Module):
    """
    DETR에서 사용되는 트랜스포머 레퍼. 인코더와 디코더 스택을 포함한다.
    """

    def __init__(self, d_model: int = 256, nhead: int = 8,
                 num_encoder_layers: int = 6,
                 num_decoder_layers: int = 6,
                 dim_feedforward: int = 2048,
                 dropout: float = 0.1) -> None:
        super().__init__()
        encoder_layer = TransformerEncoderLayer(d_model, nhead,
                                                dim_feedforward, dropout)
        decoder_layer = TransformerDecoderLayer(d_model, nhead,
                                                dim_feedforward, dropout)

        self.encoder = nn.ModuleList([encoder_layer for _ in range(num_encoder_layers)])
        self.decoder = nn.ModuleList([decoder_layer for _ in range(num_decoder_layers)])
        self.d_model = d_model
        self.nhead = nhead
        self.norm = nn.LayerNorm(d_model)

    def forward(self, src: Tensor, mask: Optional[Tensor],
               query_embed: Tensor, pos_embed: Tensor) -> Tensor:
        bs, c, h, w = src.shape
        src = src.flatten(2).permute(2, 0, 1)
        pos_embed = pos_embed.flatten(2).permute(2, 0, 1)
        query_embed = query_embed.unsqueeze(1).repeat(1, bs, 1)
        if mask is not None:
            mask = mask.flatten(1)

        memory = src
        for layer in self.encoder:
            memory = layer(memory, src_key_padding_mask=mask, pos=pos_embed)

        tgt = torch.zeros_like(query_embed)
        intermediate = []
        for layer in self.decoder:
            tgt = layer(tgt, memory,
                      memory_key_padding_mask=mask,
                      pos=pos_embed,
                      query_pos=query_embed)
            intermediate.append(self.norm(tgt.transpose(0, 1)))

        return torch.stack(intermediate)
```



## 1) BackBone

```
def build_rt_detr_backbone(train_backbone: bool = False) -> nn.Module:
    """
    ResNet-50를 기반으로 layer2/layer3/layer4 피처를 반환한다.
    """
    kwargs = {}
    if _RESNET50_DEFAULT_WEIGHTS is not None:
        kwargs["weights"] = _RESNET50_DEFAULT_WEIGHTS
    else:
        kwargs["pretrained"] = True

    backbone = resnet50(**kwargs)
    _replace_bn_with_frozen(backbone)

    return_layers = {"layer2": "0", "layer3": "1", "layer4": "2"}
    backbone = IntermediateLayerGetter(backbone, return_layers=return_layers)

    if not train_backbone:
        for param in backbone.parameters():
            param.requires_grad_(False)

    return backbone
```

## 2) Hybrid Encoder

```
class HybridEncoder(nn.Module):
    def __init__(self, in_channels: Sequence[int], hidden_dim: int,
                 nhead: int = 8, num_encoder_layers: int = 6,
                 dim_feedforward: int = 2048, dropout: float = 0.1) -> None:
        super().__init__()
        self.input_proj = nn.ModuleList([
            ConvBNAct(in_ch, hidden_dim, kernel_size=1)
            for in_ch in in_channels])
        self.position_embedding = PositionEmbeddingSine(hidden_dim // 2)
        self.level_embed = nn.Parameter(torch.randn(len(in_channels), hidden_dim))
        self.encoder_layers = nn.ModuleList([
            TransformerEncoderLayer(hidden_dim, nhead, dim_feedforward, dropout)
            for _ in range(num_encoder_layers)
        ])
        self.hidden_dim = hidden_dim

    def forward(self, features: List[Tensor], masks: List[Tensor]) -> Tuple[Tensor, Tensor, Tensor]:
        src_flatten: List[Tensor] = []
        mask_flatten: List[Tensor] = []
        pos_flatten: List[Tensor] = []

        for idx, (feature, mask) in enumerate(zip(tuple[Tensor, Tensor](features, masks))):
            proj = self.input_proj[idx](feature)
            pos = self.position_embedding(proj, mask)

            b, c, h, w = proj.shape
            proj = proj.flatten(2).permute(2, 0, 1) # (hw, B, C)
            pos = pos.flatten(2).permute(2, 0, 1)
            pos = pos + self.level_embed[idx].view(1, 1, -1)
            src_flatten.append(proj)
            pos_flatten.append(pos)
            mask_flatten.append(mask.flatten(1))

        src = torch.cat(src_flatten, dim=0)
        pos = torch.cat(pos_flatten, dim=0)
        mask = torch.cat(mask_flatten, dim=1)

        memory = src
        for layer in self.encoder_layers:
            memory = layer(memory,
                           src_key_padding_mask=mask,
                           pos=pos)

        return memory, mask, pos
```

# References

Zhao, Yian, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. "DETRs Beat YOLOs on Real-time Object Detection." arXiv preprint arXiv:2304.08069, 2024.

Carion, Nicolas, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. "End-to-End Object Detection with Transformers." In Computer Vision – ECCV 2020.

Redmon, Joseph, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. "You Only Look Once: Unified, Real-Time Object Detection." In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Redmon, Joseph, and Ali Farhadi. 2018. "YOLOv3: A Incremental Improvement." arXiv preprint arXiv:1804.02767.

Hussain, Muhammad. 2024. "YOLOv5, YOLOv8 and YOLOv10: The Go-To Detectors for Real-time Vision."

# 감사합니다.

## RT-DETR Paper Review

연구원 김재현  
E. [jh.kim@zipida.com](mailto:jh.kim@zipida.com)